Machine Learning 2026 — Assignment 4

Roll number: 2024514

Rendering and evidence note. This revision embeds a Unicode font throughout the PDF (including tables and mathematical symbols) to prevent missing-glyph black squares/dots. Graphs are embedded as PNG images. The measured results below come from executed notebook cells. CIFAR-10 could not be downloaded in this runtime because network access was unavailable; P4 CIFAR, P7 and P8 are therefore marked pending, not fabricated.

Execution summary: P1, P2, P3, Wine data preparation, P5, P6, and P9 were executed. P5's forward-pass unpacking issue was corrected. All notebook code cells pass Python syntax parsing.

P1. Activation functions and derivatives

Five activations and exact derivatives were implemented. Central differences used $\varepsilon = 10^{-6}$ at $z = [-3, -1.5, -0.5, 0.5, 1.5, 3]$.

Function	Maximum absolute error	Check
Sigmoid	4.8313×10^{-11}	PASS
tanh	1.1524×10^{-11}	PASS
ReLU	1.3978×10^{-10}	PASS
Leaky ReLU	1.3978×10^{-10}	PASS
Softplus	8.8754×10^{-11}	PASS

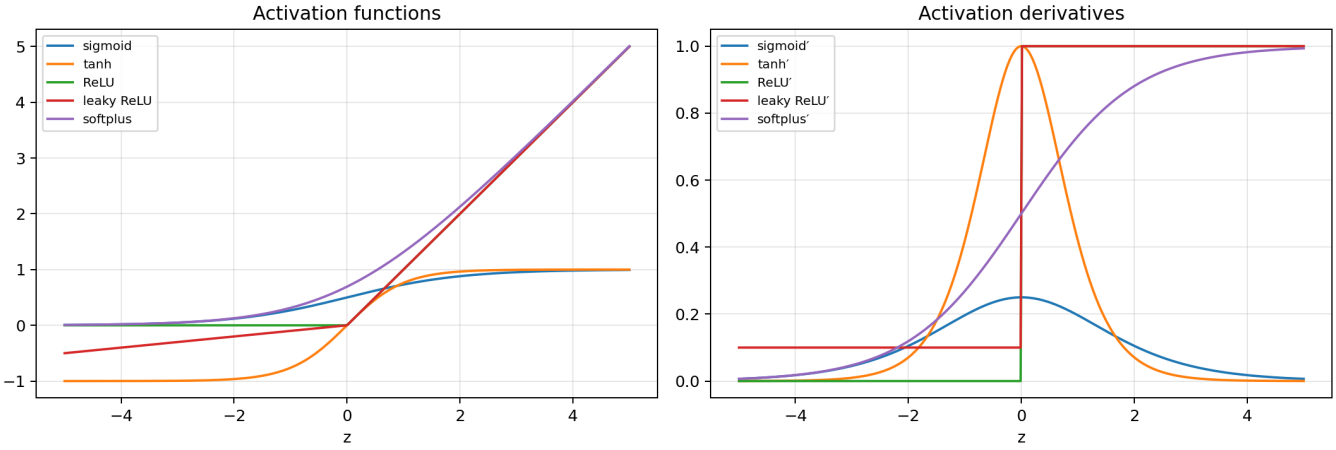


Figure 1. Activation functions and their derivatives over 400 equally spaced points in [−5, 5].

Maximum derivative on the grid: sigmoid 0.249990; tanh 0.999843; ReLU 1.000000; leaky ReLU 1.000000; softplus 0.993307.

z	sigmoid′	tanh′	ReLU′	leaky ReLU′	softplus′
−2	0.104994	0.070651	0	0.1	0.119203
0	0.250000	1.000000	0	0.1	0.500000
2	0.104994	0.070651	1	1.0	0.880797

Checkpoint: Sigmoid is the only listed activation with derivative bounded above by 1/4. Repeated multiplication by small derivatives can cause vanishing gradients. Differentiating $\tanh(z) = 2\sigma(2z) - 1$ gives $\tanh'(z) = 4\sigma'(2z)$.

P2. Backpropagation by hand

Quantity	Computed value
z	(1.100000, −0.100000)
h = tanh(z)	(0.800499, −0.099668)
u	0.620167
$\hat{y}$	0.650256
Loss	0.430388
$\partial L/\partial u$	−0.349744
$\partial L/\partial w_2$	(−0.279969, 0.034858)
$\partial L/\partial b_2$	−0.349744
$\partial L/\partial h$	(−0.209846, 0.139897)
$\partial L/\partial z$	(−0.075377, 0.138508)
$\partial L/\partial W_1$	[[-0.075377, -0.150754], [0.138508, 0.277015]]
$\partial L/\partial b_1$	(−0.075377, 0.138508)

One gradient-descent step with $\eta = 0.5$ gives loss 0.283521, lower than 0.430388. No parameter has exactly zero gradient for the supplied values.

Handwriting requirement: The assignment says P2 must be submitted handwritten. This typed solution is provided for reference; copy it by hand if literal handwriting is required.

P3. Softmax and cross-entropy

For logits [[2.0, 1.0, 0.1]] and label [1], the probabilities are [0.659001, 0.242433, 0.098566] and the cross-entropy is 1.417030.

Test	Measured output	Status
Random 5 × 4 gradient check	Maximum absolute error 2.3605×10^{-10}	PASS
Gradient row sums	All approximately zero	PASS
Shift invariance (+123.4)	Maximum difference 1.5543×10^{-15}	PASS
Naive softmax at [1000,1001,1002]	nan	Expected overflow
Stable softmax	[0.090031, 0.244728, 0.665241]	PASS

Subtracting the row maximum does not change softmax because the common exponential factor cancels. Each gradient row sums to zero because the probability row and one-hot target row each sum to one.

P4. Data pipelines

Wine: The seeded split contains 142 training and 36 validation examples, with 13 features. Standardised training feature means are approximately zero and standard deviations are 1.

CIFAR-10: Notebook code implements the prescribed 10,000-image seeded sample, 8,000/2,000 train-validation split, first 2,000 official test images, training-only channel statistics, channel-wise normalisation, channel-first flattening, class grid and class counts. Actual CIFAR outputs are unavailable because the download failed.

Concept checks: Held-out data must not determine preprocessing statistics, because that leaks validation/test information. Flattening discards explicit spatial neighbourhood structure; an MLP must learn pixel relationships without convolutional weight sharing.

P5. NumPy MLP forward pass on Wine

Item	Measured output
Architecture	13–16–8–3
Parameter count (formula)	387
Parameter count (array entries)	387
Layer 1 Z/A shape (5 examples)	(5, 16)
Layer 2 Z/A shape (5 examples)	(5, 8)
Layer 3 Z/A shape (5 examples)	(5, 3)
Initial training loss	1.726971
ln(3)	1.098612
Loss after last-layer weights × 0.01	1.101565
Vectorisation maximum difference	6.6613×10^{-16} — PASS

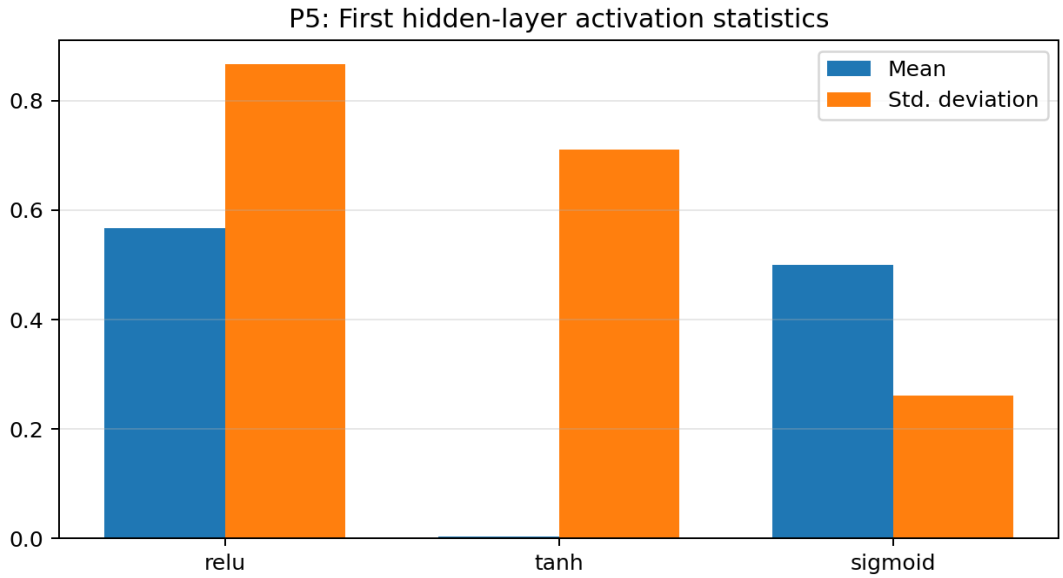


Figure 2. First hidden-layer activation means and standard deviations for the three activation choices.

Activation	Mean	Standard deviation
ReLU	0.567907	0.867025
tanh	0.004109	0.711392

Activation	Mean	Standard deviation
sigmoid	0.500716	0.261124

The initial loss is above ln(3), so the initial predictions are not uniform. Scaling the last-layer weights down reduces logit magnitudes and moves the loss close to ln(3). The output layer uses logits because softmax is combined with cross-entropy stably. Sigmoid's positive, non-zero-centred activations can make optimisation less well-conditioned.

P6. Backward pass and gradient checking

Experiment	Measured output
Correct backward pass, worst sampled relative error	3.9418×10^{-8}
8-sample overfit test	Loss < 0.01 reached in 19 steps
Bug 1: omit division by batch size	Worst relative error = 1.000000
Bug 2: use Z instead of A in dW	Worst relative error = 1.000000
Bug 3: omit activation derivative	Worst relative error = 1.000000

Central differences reduce leading truncation error compared with one-sided differences. A buggy gradient can still reduce loss temporarily if it retains a partially useful direction. The overfit test checks whether the whole training implementation can fit a tiny batch, beyond local derivative correctness.

P7. NumPy MLP on CIFAR-10

Notebook code includes the 3072–256–128–10 He-initialised MLP, 64-image overfit test, 15-epoch mini-batch training with momentum, train/validation curves, best-validation checkpoint, first-layer weight visualisations, confusion matrix, per-class accuracy and $\lambda = 10^{-3}$ comparison.

Pending: actual losses, accuracies, confusion matrix, class ranking, weight visualisations and regularisation comparison require CIFAR-10. They have not been invented.

P8. Gradient flow in deep PyTorch networks

Notebook code builds ten-hidden-layer networks for sigmoid/ReLU and Xavier/He initialisation, records mean absolute weight gradients, uses forward hooks to measure linear-layer output standard deviations, and compares sigmoid-Xavier with and without batch normalisation.

Pending: P8 requires the CIFAR-10 batch from P4, which was unavailable.

P9. Bonus — PyTorch autograd versus NumPy backpropagation

Bonus test	Measured result	Status
Maximum parameter-gradient difference	2.7756×10^{-17}	PASS ($<10^{-9}$)
Maximum dZ differences	3.4694×10^{-18}; 3.4694×10^{-18}; 1.3878×10^{-17}	PASS
Gradient doubling after two backward calls	Difference from 2× gradient = 0	PASS
no_grad output requires_grad	False	PASS
First-layer gradient after detach	None	PASS

Autograd graph traversal printed AddBackward0, MulBackward0, SumBackward0, NllLossBackward0, LogSoftmaxBackward0, MmBackward0, ReluBackward0 and AccumulateGrad nodes. Cross-entropy accepts logits for stable log-softmax computation. Gradients accumulate by default; detach is useful when intentionally stopping gradient flow.

P10 and final submission notes

The uploaded assignment PDF contains nine pages and ends at P9 (Bonus). Although its overview mentions P4–P10, no P10 prompt or marking criteria are present.

Before submission: run CIFAR-dependent sections where CIFAR-10 is available; insert actual P4/P7/P8 outputs; run the notebook top-to-bottom from a fresh kernel; handwrite P2 if required; and confirm whether a separate P10 page exists.

ZIP structure: HW4_2024514.zip contains exactly 2024514_HW4.ipynb and 2024514_HW4_report.pdf. No data folder is included.